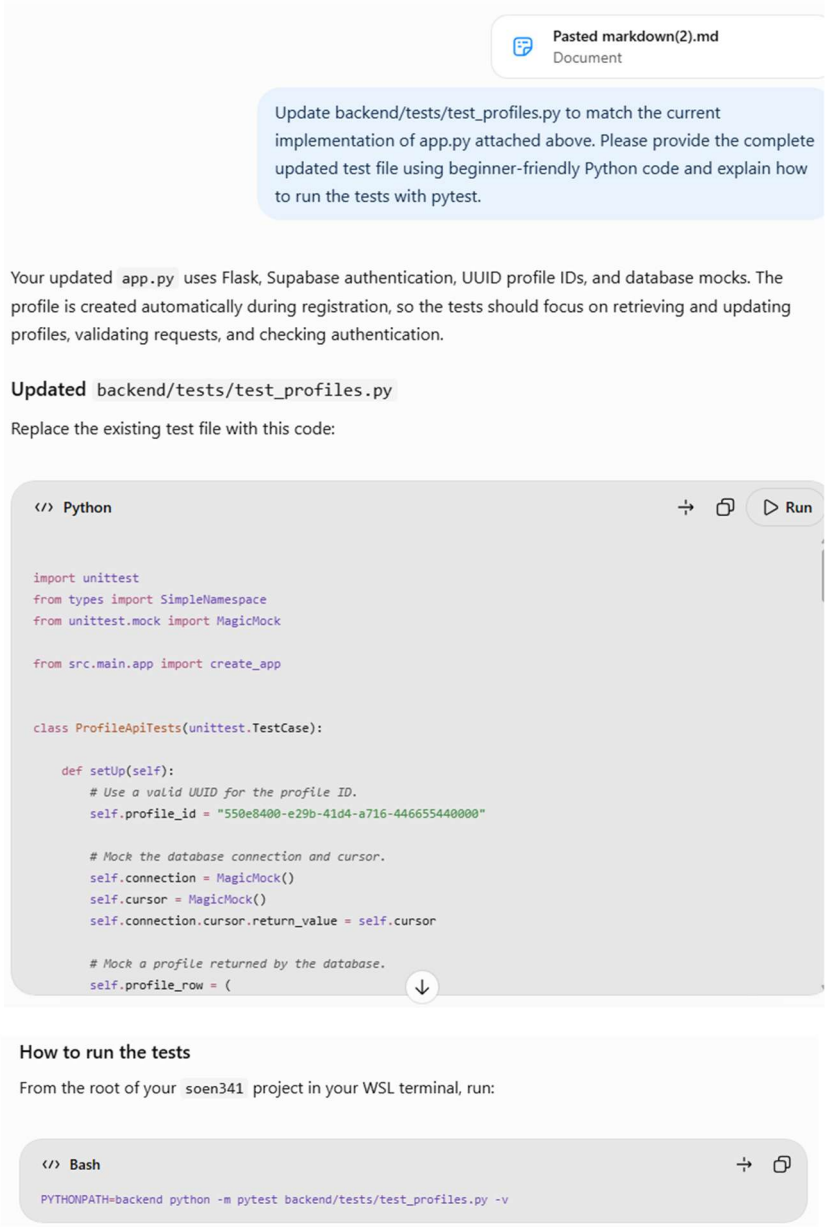
Ariana Giami, ID: 40210332

AI Logs - SOEN341 - Sprint 2

Task 1: Update test_profiles.py to match the updated implementation in app.py.

Purpose of AI Use: To help update the existing unit tests so they correctly reflect the updated profile API's routes, authentication requirements, UUID-based profile IDs, and database behavior.

Prompt/Response:



Pasted markdown(2).md
Document

Update backend/tests/test_profiles.py to match the current implementation of app.py attached above. Please provide the complete updated test file using beginner-friendly Python code and explain how to run the tests with pytest.

Your updated `app.py` uses Flask, Supabase authentication, UUID profile IDs, and database mocks. The profile is created automatically during registration, so the tests should focus on retrieving and updating profiles, validating requests, and checking authentication.

Updated `backend/tests/test_profiles.py`

Replace the existing test file with this code:

Python Run

```python
import unittest
from types import SimpleNamespace
from unittest.mock import MagicMock

from src.main.app import create_app


class ProfileApiTests(unittest.TestCase):

    def setUp(self):
        # Use a valid UUID for the profile ID.
        self.profile_id = "550e8400-e29b-41d4-a716-446655440000"

        # Mock the database connection and cursor.
        self.connection = MagicMock()
        self.cursor = MagicMock()
        self.connection.cursor.return_value = self.cursor

        # Mock a profile returned by the database.
        self.profile_row = (
```

How to run the tests

From the root of your `soen341` project in your WSL terminal, run:

Bash

```bash
PYTHONPATH=backend python -m pytest backend/tests/test_profiles.py -v
```

AI-Suggested Content: AI generated 7 unit tests covering profile retrieval, profiles not found, successful profile updates, missing required fields, invalid JSON, and authentication requirements.

Validation: The updated tests were run using “pytest”. All seven tests passed, confirming that the test cases matched the expected behavior of the profile API.

Decision: The suggested test code was accepted and used in test_profiles.py because it aligned with the updated app.py implementation and successfully passed the tests.

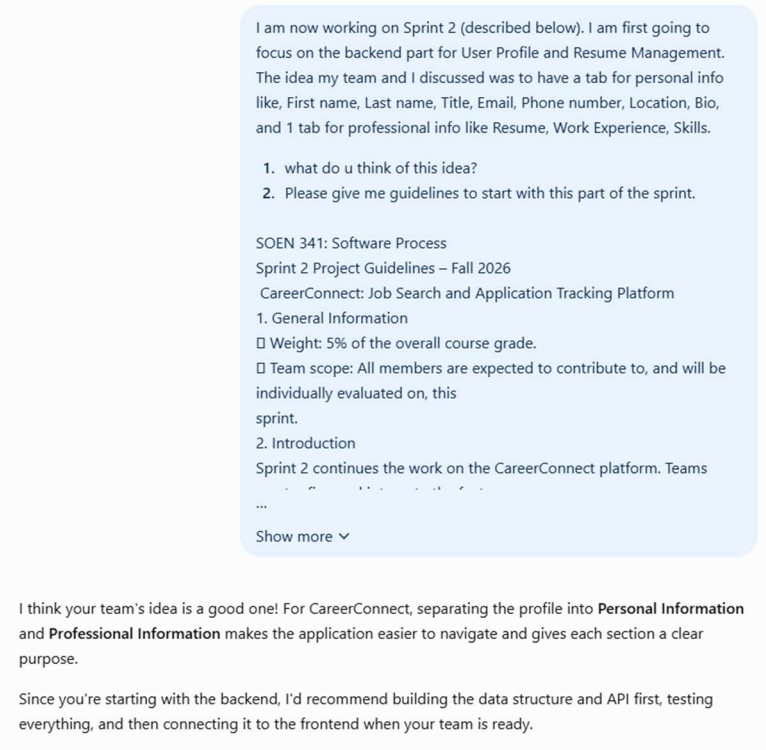
Reflection: AI helped save time by generating test cases and identifying the authentication.

Responsible Person: Ariana Giami

Task 2: Guidance for Sprint 2.

Purpose of AI Use: To provide guidance for the backend part of User Profile and Resume Management for Sprint 2.

Prompt/Response:



1. Thoughts on your two-tab idea

Tab 1 — Personal Information

Store the information that identifies the job seeker and helps recruiters contact them.

- First name
- Last name
- Professional title
- Email
- Phone number
- Location
- Bio

Backend focus: retrieve and update the user's existing profile.

Tab 2 — Professional Information

Store the information that helps the job seeker present their qualifications.

- Resume upload, viewing, replacement, and deletion
- Work experience
- Skills

Backend focus: manage resume files and create, retrieve, update, and delete work experience and skills.

One important recommendation

Even though the frontend has two tabs, **I would not put everything into one database table.**

Your existing profile API already handles personal information. Resumes, work experience, and skills have different data structures and may need separate database tables or storage.

For example:

Component	Suggested storage
Personal information	Existing `profiles` table
Resume files	Supabase Storage, with file metadata in a `resumes` table
Work experience	`work_experience` table
Skills	`skills` table, linked to the user

This is a suggested design, not a requirement stated in the sprint guidelines. It helps keep your backend organized and easier to maintain.

One additional detail: since your existing profile has a `title` field, you can use it for the user's professional title rather than creating a duplicate field.

2. Guidelines to start your backend work

I recommend following these steps in order. This way, you won't start coding before deciding what your backend needs to support.

Step 1 — Review what you already have

Before creating new files, review your existing Sprint 1 implementation.

You already have:

- `GET /api/profiles/<profile_id>` — retrieves a user's profile.
- `PUT /api/profiles/<profile_id>` — updates a user's profile.
- Authentication checks to ensure a user can only access their own profile.
- Unit tests in `backend/tests/test_profiles.py`.

Step 2 — Break the feature into smaller tasks

Your sprint guidelines require user stories and tasks in GitHub. I would break your User Profile and Resun
Management feature into the following user stories.

US — View and Edit Personal Profile

Build on your existing profile API.

- Task 1: Review the existing profile schema and API.
- Task 2: Verify retrieval and updating of profile fields.
- Task 3: Add or update tests for required fields and validation.

US — Manage Resumes

New backend functionality.

- Task 1: Decide how resume files and metadata will be stored.
- Task 2: Implement resume upload.
- Task 3: Implement resume listing and viewing.
- Task 4: Implement resume replacement and deletion.
- Task 5: Write tests for the resume operations.

US — Manage Work Experience

New backend functionality.

- Task 1: Design the work experience database table.
- Task 2: Implement create, read, update, and delete operations.
- Task 3: Test validation and user ownership.

US — Manage Skills

New backend functionality.

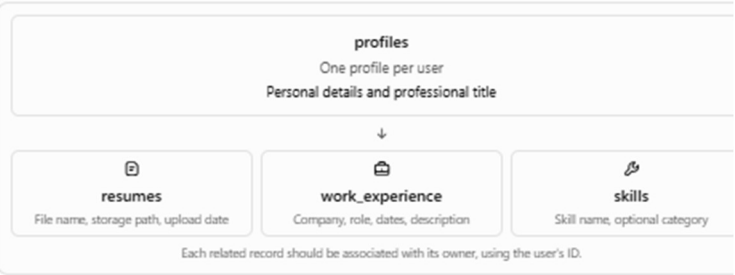
- Task 1: Design how skills will be stored.
- Task 2: Implement adding, listing, and removing skills.
- Task 3: Write tests for skill management.

These are proposed stories and tasks. Add them to your GitHub project, assign owners, and adjust the sc
with your teammates before implementation.

Step 3 — Design your database before coding

You don't need to implement all the tables at once. First, understand the relationships.

Here is one possible design:



A few decisions to discuss with your team:

- **Resumes:** Will a user be allowed to keep multiple resumes, or only one active resume?
- **Work experience:** Which fields are required? I suggest company, job title, start date, end date, and description. Allow an empty end date for a current job.
- **Skills:** Can a user add the same skill more than once? Usually, duplicate skills should be prevented.
- **Ownership:** How will every record be linked to the authenticated user?

Since you are already using Supabase, a reasonable option is to use **Supabase Storage for the actual resume files** and PostgreSQL for the resume metadata. Avoid storing an entire PDF directly in a normal database text field.

Step 4 — Decide on your API endpoints

Your frontend tabs will eventually call backend endpoints. For example:

Operation	Suggested endpoint
View personal profile	`GET /api/profiles/<profile_id>`
Edit personal profile	`PUT /api/profiles/<profile_id>`
List resumes	`GET /api/profiles/<profile_id>/resumes`
Upload resume	`POST /api/profiles/<profile_id>/resumes`
Replace resume	`PUT /api/profiles/<profile_id>/resumes/<resume_id>`
Delete resume	`DELETE /api/profiles/<profile_id>/resumes/<resume_id>`
List work experience	`GET /api/profiles/<profile_id>/experience`
Add work experience	`POST /api/profiles/<profile_id>/experience`
Update work experience	`PUT /api/profiles/<profile_id>/experience/<experience_id>`
Delete work experience	`DELETE /api/profiles/<profile_id>/experience/<experience_id>`
List skills	`GET /api/profiles/<profile_id>/skills`
Add skill	`POST /api/profiles/<profile_id>/skills`
Remove skill	`DELETE /api/profiles/<profile_id>/skills/<skill_id>`

These are proposed routes; they don't need to be your final naming convention. Agree on them with your team before implementing them.

Security reminder: Every endpoint must enforce authentication and ownership. For resume uploads, validate allowed file types and file sizes, and keep files private unless your team deliberately designs a secure sharing mechanism.

AI-Suggested Content: Suggested a two-section profile structure and backend implementation plan for personal information, resumes, work experience, and skills.

Validation: Reviewed the suggestions against the existing backend structure and Sprint 2 requirements.

Decision: Accepted the proposed organization and used the suggestions to guide backend implementation decisions, including file validation, authentication, and data management.

Reflection: AI assistance helped clarify the feature requirements and break the backend work into manageable tasks.

Responsible Person: Ariana Giami

Task 3: Resume Management

Purpose of AI Use: Assist with implementing resume-management endpoints in the Flask backend.

Prompt/Response:

Add resume endpoints and update the create_ap() function to reflect resume upload, resume retrieval, replacing an existing resume, and deleting a resume (only in .pdf or .doc files, so validation is required as well here).

Yes! We can start by adding the resume-management functionality in a way that follows your existing Flask code and mocked-test setup. We'll keep your current profile endpoints unchanged and make the storage bucket name configurable, so you can confirm the actual name later.

The initial implementation will support:

- `POST` — upload a resume.
- `GET` — retrieve resume details and a temporary download link.
- `PUT` — replace the existing resume.
- `DELETE` — delete the resume.
- Authentication and ownership checks, plus `.pdf` and `.doc` file validation.

One important detail: the code can be prepared now, but uploading and downloading files will require the actual Supabase Storage bucket and database table to be configured.

1. Add the resume endpoints to `app.py`

Open `backend/src/main/app.py` in VS Code.

Step 1 — Add one import

At the top of the file, add:

Python Run

```python
from werkzeug.utils import secure_filename
```

Step 2 — Add the bucket configuration

Inside `create_app()`, alongside your other `app.config` settings, add:

Python Run

```python
RESUME_BUCKET=os.getenv("SUPABASE_RESUME_BUCKET", "resumes"),
RESUME_MAX_SIZE=10 * 1024 * 1024,  # 10 MB
RESUME_SIGNED_URL_SECONDS=3600,    # 1 hour
```

This makes the bucket name configurable. For now, the code assumes the bucket is named `resumes`.

Step 3 — Add the routes

Paste the following code **inside** `create_app()` **and immediately before** `return app`. Do not put it at the very bottom of the file outside the function.

Python Run

```python
# Resume management

RESUME_COLUMNS = [
    "id",
    "profile_id",
    "file_name",
    "storage_path",
    "content_type",
    "file_size",
    "uploaded_at",
]

ALLOWED_RESUME_TYPES = {
    ".pdf": "application/pdf",
    ".doc": "application/msword",
}

def resume_to_dict(row):
    """Convert a database resume row into a JSON-friendly dictionary."""
    return dict(zip(RESUME_COLUMNS, row))
```

2. How the API will work

All four endpoints use the same URL, with different HTTP methods.

Method	Endpoint	Purpose
`POST`	`/api/profiles/{profile_id}/resume`	Upload the first resume
`GET`	`/api/profiles/{profile_id}/resume`	Get resume details and a temporary download link
`PUT`	`/api/profiles/{profile_id}/resume`	Replace the existing resume
`DELETE`	`/api/profiles/{profile_id}/resume`	Delete the resume

AI-Suggested Content: Add upload, retrieval, replacement, and deletion endpoints with .pdf and .doc validation, authentication, and Supabase Storage integration.

Validation: Reviewed the suggested code and run syntax checks and mocked tests.

Decision: Used the suggestions as a starting point while preserving existing profile functionality.

Reflection: AI helped structure the implementation.

Responsible Person: Ariana Giami

Task 4: Job Search and Job Listing Interface

Purpose of AI Use: Assist with implementing job search and job listing endpoints in the Flask backend.

Prompt/Response:

I am now working on the backend for Job Search and Job Listing Interface. The tasks needed to be performed are the following: to display available job opportunities, provide search and filtering capabilities based on keywords, location, or category (to also have a minimum salary that job seekers can indicate if they want a certain minimum salary), and allow users to view detailed job descriptions. Give step by step instructions on how to apply these features.

Worked for 18s >

For CareerConnect, I recommend implementing the **Job Search and Job Listing Interface** in small steps, using your existing Flask backend and PostgreSQL database setup. The goal is to let job seekers browse available jobs, search and filter them, and open a job to see its full description.

We'll build the backend first, then test each endpoint before connecting it to the frontend.

Overview of what you'll implement

Feature	Backend responsibility
Display available jobs	Return a list of job postings
Search by keyword	Match words against job titles, descriptions, or company names
Filter by location	Return jobs in a selected location
Filter by category	Return jobs in a selected category
Minimum salary	Return jobs whose salary meets the user's chosen minimum
View job details	Return the full information for one job posting

Step 1 — Check the existing database and code

Before writing anything, make sure the database structure matches what the backend will query. Your team may already have a `jobs` or `job_listings` table, especially if another teammate is working on recruiter job posting.

In VS Code:

1. Open your `soen341` project.
2. Look for database schema files, SQL setup scripts, or migrations.
3. Check `backend/src/main/app.py` for any existi ob-related routes.
4. Check whether your teammate has already implemented creating or editing job postings.

Step 2 — Decide on the API endpoints

You can implement this feature with **two main GET endpoints**.

`GET` `/api/jobs`

Returns available jobs and applies any search filters provided.

Example: `/api/jobs?keyword=software&location=Montreal&min_salary=60000`

`GET` `/api/jobs/<job_id>`

Returns the complete details of one job posting.

Example: `/api/jobs/12`

You don't need separate endpoints for every filter. The first endpoint can handle all of them together.

Step 3 — Implement the job listing endpoint

Open `backend/src/main/app.py` and find the section where your other routes are defined. Add the new route inside `create_app()`, before `return app`.

The example below assumes your database table is named `jobs`. **Adjust the table and column names to match your actual schema before running it.**

How this works

- With no filters, it returns all published jobs.
- `keyword` searches the title, description, and company name.
- `location` matches a location, even if the capitalization differs.
- `category` filters by the selected category.
- `min_salary` filters by the posted minimum salary.
- Results are ordered newest first.

The `%s` placeholders and separate `params` list are important: they let PostgreSQL safely handle the search values rather than inserting user input directly into SQL.

Before using this code: verify that your actual table has the listed columns. Also, if your schema uses another way to identify published jobs, update `status = %s`. If salary fields can be `NULL`, decide with your team whether jobs without salary information should appear when a minimum salary filter is used.

Step 4 — Implement the job details endpoint

This endpoint retrieves a single posting by its ID. Use the same columns as your actual database schema.

Python Run

```python
@app.get("/api/jobs/<int:job_id>")
def get_job_details(job_id):
    connection = None

    try:
        connection = get_connection()
        cursor = connection.cursor()

        cursor.execute(
            """
            SELECT id, title, company_name, location, category,
                   description, salary_min, salary_max, created_at
            FROM jobs
            WHERE id = %s AND status = %s
            """,
            (job_id, "published"),
        )

        row = cursor.fetchone()

        if row is None:
```

When a job seeker selects a listing, the frontend can call this endpoint to retrieve the complete description, requirements, salary range, and other details included in the schema.

Step 5 — Add tests

Create a file named `backend/tests/test_jobs.py`, following the same structure as your existing `test_resumes.py` tests.

Start with these test cases:

Test	Expected result
Retrieve jobs without filters	`200 OK`
Search by keyword	Only matching jobs are returned
Filter by location	Only jobs matching the location are returned
Filter by category	Only jobs in that category are returned
Filter by minimum salary	Jobs below the threshold are excluded
Combine multiple filters	All filters apply together
Pass an invalid minimum salary	`400 Bad Request`
Request an existing job ID	`200 OK`
Request a nonexistent job ID	`404 Not Found`

AI-Suggested Content: Features like displaying jobs, searching by keyword, filtering by location or category we're suggested.

Validation: Reviewed the suggested code and run syntax checks and mocked tests.

Decision: Used the suggestions as a starting point while preserving existing profile functionality.

Reflection: AI helped structure the implementation.

Responsible Person: Ariana Giami